

Ex. No: 12 Page Replacement Algorithms

PROGRAM A: FIFO PAGE REPLACEMENT

C PROGRAM

```
#include <stdio.h>

int main()
{
    int pages[50], frames[10];

    int n, f, i, j, k = 0;

    int fault = 0, found;

    printf("Enter Number of Pages: ");

    scanf("%d", &n);

    printf("Enter Reference String:\n");

    for(i = 0; i < n; i++)

        scanf("%d", &pages[i]);

    printf("Enter Number of Frames: ");

    scanf("%d", &f);

    for(i = 0; i < f; i++)

        frames[i] = -1;

    for(i = 0; i < n; i++)

    {

        found = 0;

        for(j = 0; j < f; j++)

        {

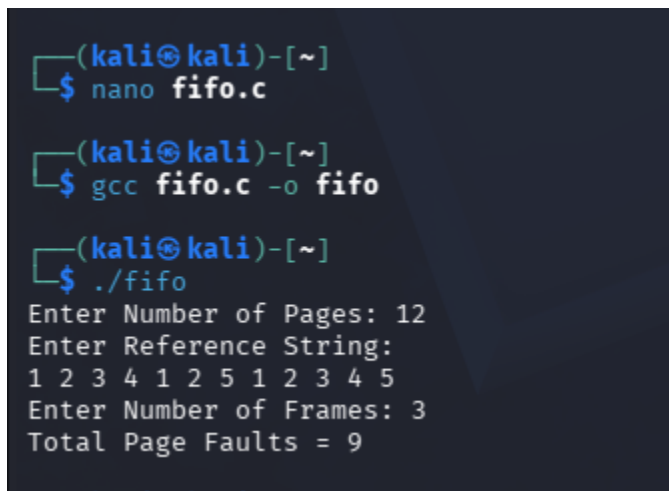
            if(frames[j] == pages[i])
```

```

found = 1;
break;
}
}
if(found == 0)
{
frames[k] = pages[i];
k = (k + 1) % f;
fault++;
}
}
printf("Total Page Faults = %d\n", fault);
return 0; }

```

Output:



```

(kali㉿kali)-[~]
$ nano fifo.c

(kali㉿kali)-[~]
$ gcc fifo.c -o fifo

(kali㉿kali)-[~]
$ ./fifo
Enter Number of Pages: 12
Enter Reference String:
1 2 3 4 1 2 5 1 2 3 4 5
Enter Number of Frames: 3
Total Page Faults = 9

```

PROGRAM B: LRU PAGE REPLACEMENT

C PROGRAM

```
#include <stdio.h>

int main() {

int pages[50], frames[10], time[10];

int n, f, i, j;

int fault = 0, count = 0;

int found, pos, min;

printf("Enter Number of Pages: ");

scanf("%d", &n);

printf("Enter Reference String:
\n");


for(i = 0; i < n; i++)

scanf("%d", &pages[i]);

printf("Enter Number of Frames: ");

scanf("%d", &f);

for(i = 0; i < f; i++)

frames[i] =

-1;

for(i = 0; i < n; i++)

{

found = 0;

for(j = 0; j < f; j++)

{

if(frames[j] == pages[i])

{

count++;
```

```
time[j] = count;
found = 1;
break;
}
}
if(found == 0)
{
min = time[0];
pos = 0;
for(j = 0; j < f; j++)
{
```

69

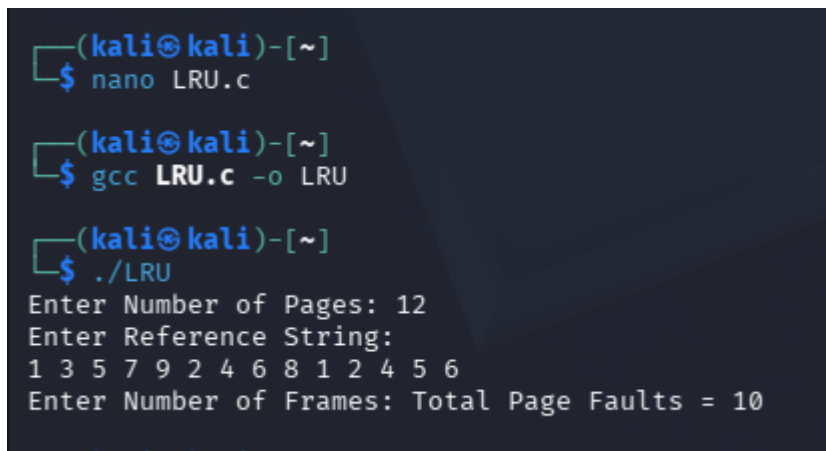
```
if(frames[j] == -1)
{
pos = j;
break;
}
if(time[j] < min)
{
min = time[j];
pos = j;
}
}
frames[pos] = pages[i];
```

```

count++;
time[pos] = count;
fault++;
}
}
printf("Total Page Faults = %d\n", fault);
return 0;
}

```

Output:



```

(kali㉿kali)-[~]
$ nano LRU.c

(kali㉿kali)-[~]
$ gcc LRU.c -o LRU

(kali㉿kali)-[~]
$ ./LRU
Enter Number of Pages: 12
Enter Reference String:
1 3 5 7 9 2 4 6 8 1 2 4 5 6
Enter Number of Frames: Total Page Faults = 10

```

SHELL SCRIPT

```

#!/bin/bash

echo "LRU Page Replacement Demonstration"

pages=(7 0 1 2 0 3 0 4 2 3 0 3 2)

echo "Reference String: ${pages[@]}"

echo "LRU Algorithm Executed"

```

Output:

```
(kali㉿kali)-[~]  
$ nano lab12-2.sh  
  
(kali㉿kali)-[~]  
$ chmod +x lab12-2.sh  
  
(kali㉿kali)-[~]  
$ bash lab12-2.sh  
LRU Page Replacement Demonstration  
Reference String: 7 0 1 2 0 3 0 4 2 3 0 3 2  
LRU Algorithm Executed
```

PROGRAM C: OPTIMAL PAGE REPLACEMENT

C PROGRAM

```
#include <stdio.h>  
  
int main()  
{  
    int pages[50], frames[10];  
  
    int n, f;  
  
    int i, j, k, pos;  
  
    int fault = 0;  
  
    int found;  
  
    printf("Enter Number of Pages: ");  
    scanf("%d", &n);  
  
    printf("Enter Reference String:  
    \n");  
  
    for(i = 0; i < n; i++)  
        scanf("%d", &pages[i]);  
  
    printf("Enter Number of Frames: ");  
    scanf("%d", &f);  
  
    for(i = 0; i < f; i++)  
        frames[i] =
```

```
-1;
for(i = 0; i < n; i++)
{
    found = 0;
    for(j = 0; j < f; j++)
    {
        if(frames[j] == pages[i])
        {
            found = 1;
            break;
        }
    }
    if(found == 0)
    {
        for(j = 0; j < f; j++)
        {
            int future = 999;
            for(k = i + 1; k < n; k++)
            {
                if(frames[j] == pages[k])

            }

            future = k;
            break;
        }
    }
}
```

```

}
if(j == 0 || future > pos)
{
    pos = future;
}
}
frames[0] = pages[i];
fault++;
}
}
printf("Total Page Faults = %d\n", fault);
return 0;
}

```

Output:

```

(kali@kali)-[~]
$ nano optimal.c

(kali@kali)-[~]
$ gcc optimal.c -o optimal

(kali@kali)-[~]
$ ./optimal
Enter Number of Pages: 5
Enter Reference String:
1 3 5 7 9 2 4 5 7
Enter Number of Frames: Total Page Faults = 5

```

SHELL SCRIPT


```
#!/bin/bash

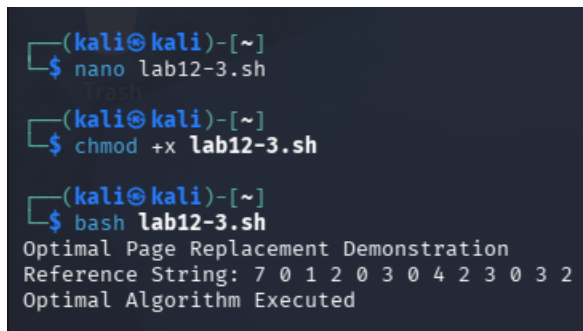
echo "Optimal Page Replacement Demonstration"

pages=(7 0 1 2 0 3 0 4 2 3 0 3 2)

echo "Reference String: ${pages[@]}"

echo "Optimal Algorithm Executed"
```

Output:



```
(kali㉿kali)-[~]
$ nano lab12-3.sh
$ chmod +x lab12-3.sh
$ bash lab12-3.sh
Optimal Page Replacement Demonstration
Reference String: 7 0 1 2 0 3 0 4 2 3 0 3 2
Optimal Algorithm Executed
```

The image shows a terminal window on a Kali Linux system. The user creates a file named 'lab12-3.sh' using 'nano', makes it executable with 'chmod +x', and then runs it with 'bash'. The script's output is displayed in the terminal, showing the reference string and the execution of the optimal algorithm.